



American International University- Bangladesh (AIUB)

Faculty of Engineering (EEE)

Course Name:	Microprocessor and Embedded Systems	Course Code:	COE3104
Semester:	Summer 2024-2025	Sec:	K
Lab Instructor:	Md. Ali Noor		

Experiment No:	07
Experiment Name:	Interfacing the Arduino with an external sensor using serial communication protocol for implementing an obstacle detection system.

Group Members	Name	ID
1.	Basudeb Kundu	23-50856-1
2.	Prohlad Chandra Das	23-50922-1
3.	Debashis Kumar Das	23-50953-1
4.	Indronill Dutta Nill	23-50974-1
5.	Nafiur Rahman Nirob	23-50991-1

Performance Date:	3-8-25	Due Date:	07-09-25
--------------------------	--------	------------------	----------

Marking Rubrics (to be filled by Lab Instructor)

Category	Proficient [6]	Good [4]	Acceptable [2]	Unacceptable [1]	Secured Marks
Theoretical Background, Methods & procedures sections	All information, measures and variables are provided and explained.	All Information provided that is sufficient, but more explanation is needed.	Most information correct, but some information may be missing or inaccurate.	Much information missing and/or inaccurate.	
Results	All of the criteria are met; results are described clearly and accurately;	Most criteria are met, but there may be some lack of clarity and/or incorrect information.	Experimental results don't match exactly with the theoretical values and/or analysis is unclear.	Experimental results are missing or incorrect;	
Discussion	Demonstrates thorough and sophisticated understanding. Conclusions drawn are appropriate for analyses;	Hypotheses are clearly stated, but some concluding statements not supported by data or data not well integrated.	Some hypotheses missing or misstated; conclusions not supported by data.	Conclusions don't match hypotheses, not supported by data; no integration of data from different sources.	
General formatting	Title page, placement of figures and figure captions, and other formatting issues all correct.	Minor errors in formatting.	Major errors and/or missing information.	Not proper style in text.	
Writing & organization	Writing is strong and easy to understand; ideas are fully elaborated and connected; effective transitions between sentences; no typographic, spelling, or grammatical errors.	Writing is clear and easy to understand; ideas are connected; effective transitions between sentences; minor typographic, spelling, or grammatical errors.	Most of the required criteria are met, but some lack of clarity, typographic, spelling, or grammatical errors are present.	Very unclear, many errors.	
Comments:				Total Marks (Out of):	

Experiment Title

Interfacing the Arduino with an external sensor using serial communication protocol for implementing an obstacle detection system.

Objectives

The objectives of this experiment are to-

1. Write code for a simple obstacle detection system in Arduino IDE.
2. Implement a simple obstacle detection system using an Arduino microcontroller.

Theory and Methodology

Arduino is an open-source platform used for creating interactive electronics projects. It consists of both a programmable microcontroller and an Integrated Development Environment (IDE) that runs on a computer, used to write and upload code to the microcontroller board. Arduino Uno does not require an external programmer to load code; it can be done easily via USB using the Arduino IDE, which utilizes a simplified version of C++.

In this experiment, we use a sonar sensor (HCSR04) to measure the distance to an obstacle. Based on this distance, LEDs are activated to indicate how close the object is to the sensor.

The HCSR04 ultrasonic sensor operates using sonar signals to detect object distance. It can measure distances ranging from 2 cm to 400 cm with an accuracy of 0.3 cm. The sensor has four pins: VCC, GND, Trigger, and Echo. When the Trigger pin receives a 10 μ s pulse from the Arduino, it emits eight 40 kHz ultrasonic pulses. If these pulses hit an object, they reflect back and are received by the Echo pin.

The Echo pin outputs a pulse whose duration corresponds to the time it took the signal to travel to the object and back. This travel time is used to calculate the distance:

$$\text{Distance} = \frac{v \times t}{2}$$

Where:

- $v = 340 \text{ m/s}$ (speed of sound in air)
- $t = \text{travel time (in seconds)}$

The Trigger pin is connected to Arduino digital pin 11, and the Echo pin to digital pin 12. LEDs are connected to pins 2, 3, and 4 to indicate different distance thresholds. These pins are configured as outputs in the Arduino sketch. When an object enters a certain proximity, the LEDs light up to visually indicate how close it is.

The methodology involves coding in the Arduino IDE, setting up the circuit, and observing the LED behavior and serial monitor output as objects move within the sensor's range.

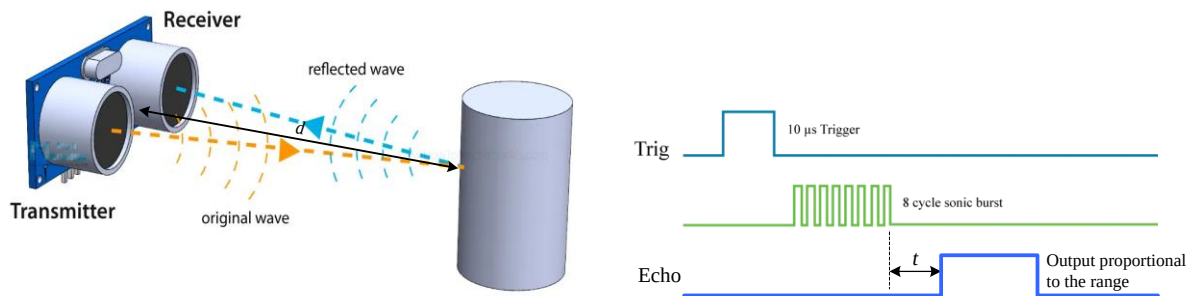


Fig1.1: Working Principle and Timing Diagram of HCSR04 Ultrasonic Sensor

Apparatus

- 1) Arduino IDE
- 2) Arduino UNO (R3) board
- 3) Sonar Sensor (HCSR04)
- 4) Breadboard
- 5) LEDs (red, green and yellow)
- 6) Resistor of 100 Ω
- 7) Jumper Wires.

Circuit Diagram:

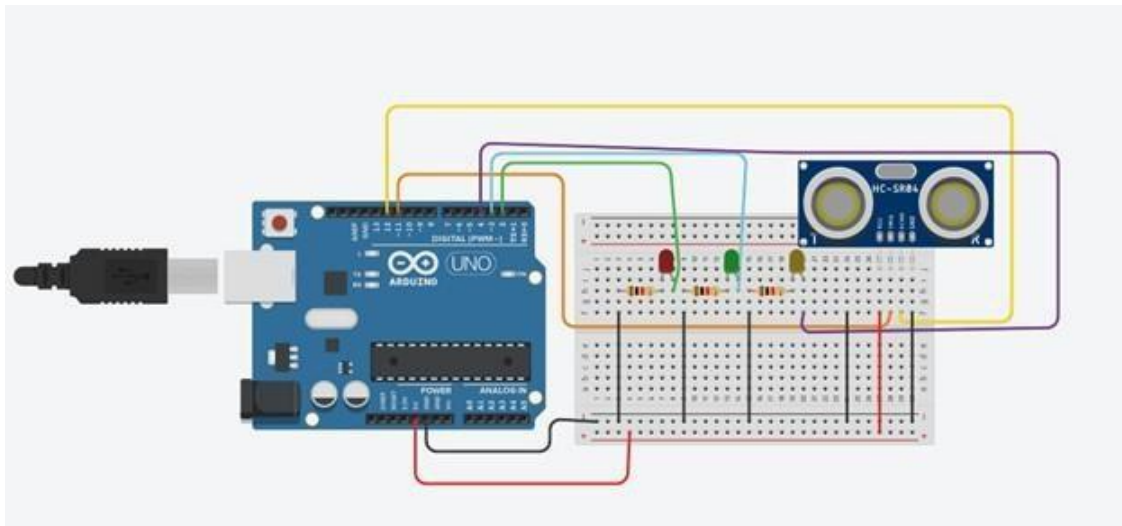


Fig1.2: Experimental setup of an obstacle detection system using Arduino

Code/Program

The following is the code for the implementation of an obstacle detection system using Arduino with the necessary code explanation:

```
// Define the pin numbers
const int trigPin = 11;
const int echoPin = 12;

// Define variables
long duration;
float distance, distanceInches, distanceThreshold;

void setup() {
  Serial.begin(9600); // Starts the serial communication
  pinMode(trigPin, OUTPUT); // Sets the trigPin as an Output
  pinMode(echoPin, INPUT); // Sets the echoPin as an Input
  pinMode(2, OUTPUT); // Sets pins 2, 3, and 4 as Output pins
  pinMode(3, OUTPUT);
  pinMode(4, OUTPUT);
}

void loop() {
  // Clears the trigPin
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);

  // Sets the trigPin on HIGH state for 10 microseconds
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // Reads the echoPin, returns the sound wave travel time in microseconds
  duration = pulseIn(echoPin, HIGH);

  // Calculates the distance in cm and inches
  distance = (duration / 2) * 0.0343 * 100; // Convert to cm
  distanceInches = distance / 2.54; // Convert to inches

  // Prints the distance on the Serial Monitor
  Serial.print("Distance = ");
  Serial.print(distance);
  Serial.print(" cm; ");
  Serial.print("Distance = ");
  Serial.print(distanceInches);
  Serial.println(" inches");
```

```

// Set threshold distance to activate LEDs
Distance Threshold = 80;

// Control LEDs based on the distance
if (distance > distanceThreshold) {
    digitalWrite(2, LOW);
    digitalWrite(3, LOW);
    digitalWrite(4, LOW);
}
else if (distance <= distanceThreshold && distance > distanceThreshold - 30) {
    digitalWrite(2, HIGH);
    digitalWrite(3, LOW);
    digitalWrite(4, LOW);
}
else if (distance <= distanceThreshold - 30 && distance > distanceThreshold -
50) {
    digitalWrite(2, HIGH);
    digitalWrite(3, HIGH);
    digitalWrite(4, LOW);
}
else if (distance <= distanceThreshold - 50 && distance > distanceThreshold -
70) {
    digitalWrite(2, HIGH);
    digitalWrite(3, HIGH);
    digitalWrite(4, HIGH);
}

delay(200); // Wait for 200 milliseconds
}

```

Experimental Circuits And Result

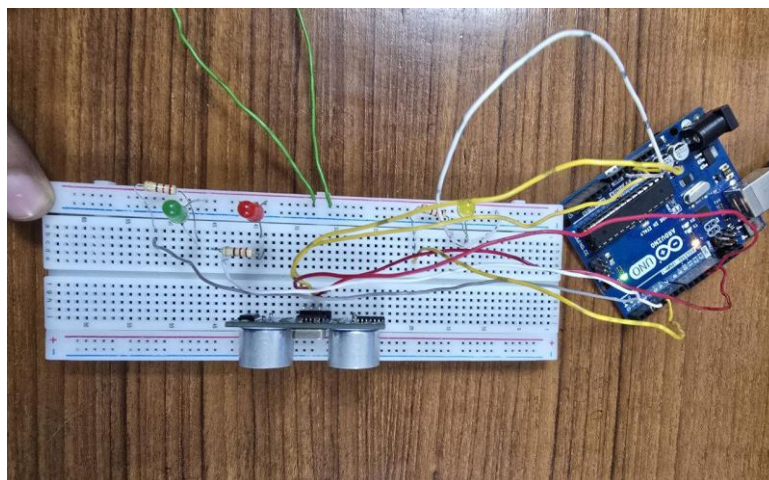


Fig2: Hardware implementation of an obstacle detection system using Arduino

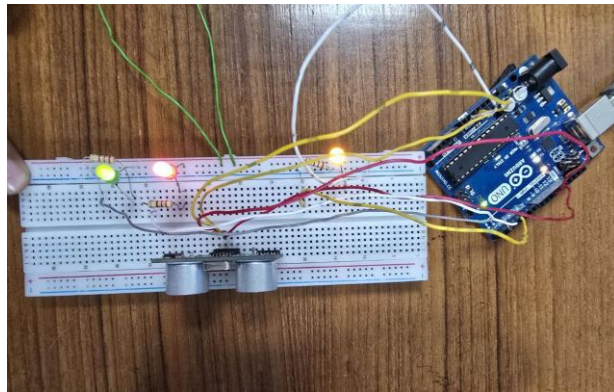


Fig3.1: ALL turned ON when a distance as at a close distance

```
10:51:26.780 -> Distance = 19.18 cm; Distance = 7.55 inches
10:51:26.968 -> Distance = 19.18 cm; Distance = 7.55 inches
10:51:27.159 -> Distance = 18.77 cm; Distance = 7.39 inches
10:51:27.394 -> Distance = 19.24 cm; Distance = 7.58 inches
10:51:27.582 -> Distance = 19.24 cm; Distance = 7.58 inches
10:51:27.769 -> Distance = 19.24 cm; Distance = 7.58 inches
10:51:28.004 -> Distance = 19.14 cm; Distance = 7.54 inches
10:51:28.190 -> Distance = 18.77 cm; Distance = 7.39 inches
10:51:28.378 -> Distance = 18.70 cm; Distance = 7.36 inches
10:51:28.613 -> Distance = 18.67 cm; Distance = 7.35 inches
10:51:28.803 -> Distance = 18.77 cm; Distance = 7.39 inches
10:51:28.992 -> Distance = 18.70 cm; Distance = 7.36 inches
10:51:29.227 -> Distance = 18.29 cm; Distance = 7.20 inches
10:51:29.417 -> Distance = 18.70 cm; Distance = 7.36 inches
10:51:29.652 -> Distance = 18.70 cm; Distance = 7.36 inches
```

Fig3.2: Serial Monitor output when the ALL LED turns ON

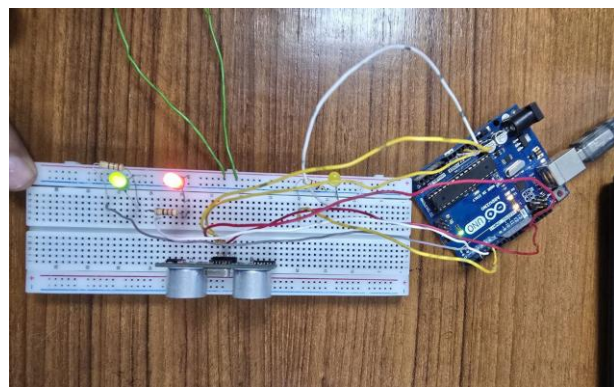


Fig4.1: Two turned ON when a distance as at a medium distance

```
10:50:41.192 -> Distance = 31.11 cm; Distance = 12.25 inches
10:50:41.382 -> Distance = 31.59 cm; Distance = 12.44 inches
10:50:41.572 -> Distance = 31.04 cm; Distance = 12.22 inches
10:50:41.808 -> Distance = 31.52 cm; Distance = 12.41 inches
10:50:41.997 -> Distance = 31.55 cm; Distance = 12.42 inches
10:50:42.187 -> Distance = 31.42 cm; Distance = 12.37 inches
10:50:42.423 -> Distance = 31.45 cm; Distance = 12.38 inches
10:50:42.611 -> Distance = 31.35 cm; Distance = 12.34 inches
10:50:42.800 -> Distance = 31.25 cm; Distance = 12.30 inches
10:50:43.036 -> Distance = 31.25 cm; Distance = 12.30 inches
10:50:43.225 -> Distance = 31.25 cm; Distance = 12.30 inches
10:50:43.416 -> Distance = 31.21 cm; Distance = 12.29 inches
10:50:43.652 -> Distance = 30.94 cm; Distance = 12.18 inches
10:50:43.842 -> Distance = 31.11 cm; Distance = 12.25 inches
10:50:44.031 -> Distance = 31.01 cm; Distance = 12.21 inches
```

Fig4.2: Serial Monitor output when the Two LED turns ON

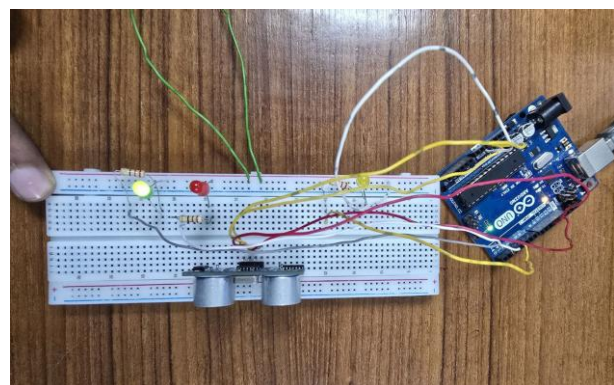


Fig5.1: One LED turned ON when a distance as at a far distance

```
10:49:37.476 -> Distance = 56.64 cm; Distance = 22.30 inches
10:49:37.712 -> Distance = 57.19 cm; Distance = 22.51 inches
10:49:37.902 -> Distance = 56.00 cm; Distance = 22.05 inches
10:49:38.138 -> Distance = 55.45 cm; Distance = 21.83 inches
10:49:38.328 -> Distance = 55.45 cm; Distance = 21.83 inches
10:49:38.520 -> Distance = 56.78 cm; Distance = 22.35 inches
10:49:38.757 -> Distance = 55.90 cm; Distance = 22.01 inches
10:49:38.945 -> Distance = 56.34 cm; Distance = 22.18 inches
10:49:39.134 -> Distance = 56.17 cm; Distance = 22.11 inches
10:49:39.371 -> Distance = 58.07 cm; Distance = 22.86 inches
10:49:39.560 -> Distance = 57.19 cm; Distance = 22.51 inches
10:49:39.796 -> Distance = 58.14 cm; Distance = 22.89 inches
10:49:39.985 -> Distance = 57.63 cm; Distance = 22.69 inches
10:49:40.174 -> Distance = 56.71 cm; Distance = 22.33 inches
10:49:40.412 -> Distance = 55.69 cm; Distance = 21.93 inches
```

Fig5.2: Serial Monitor output when the One LED turns ON

Explanation:

In this experiment, an object was placed in front of the ultrasonic sensor while the circuit was running. When the object was far within the sensor's range (over 50 cm or 20 inches), only the RED LED turned ON. As the object moved closer (30-50 cm or 7-20 inches), the YELLOW LED also turned ON. Finally, when the object was within a short range, the GREEN LED lit up. The serial monitor displayed the correct distances in both centimeters and inches, confirming the system's accuracy

Simulation Circuits And Result

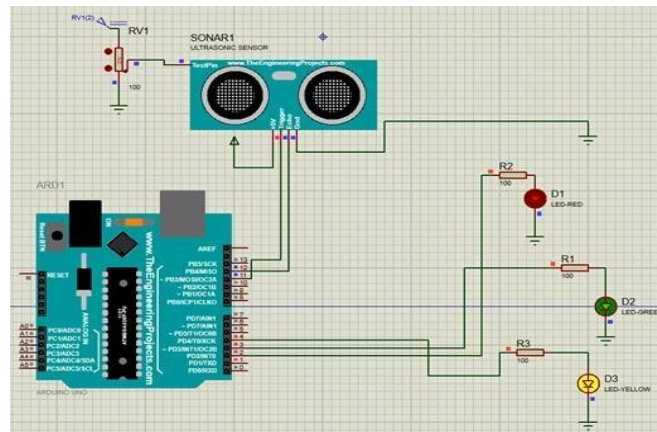


Fig6: All LED ON in Proteus Simulation when the object was in Close Distance

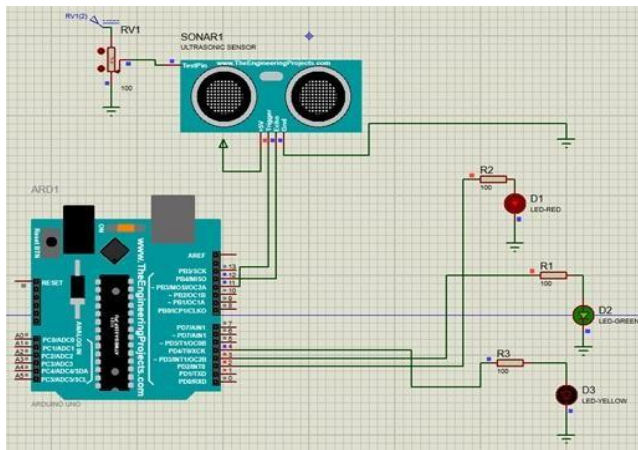


Fig7: Two LED ON in Proteus Simulation when the object was in Medium Distance

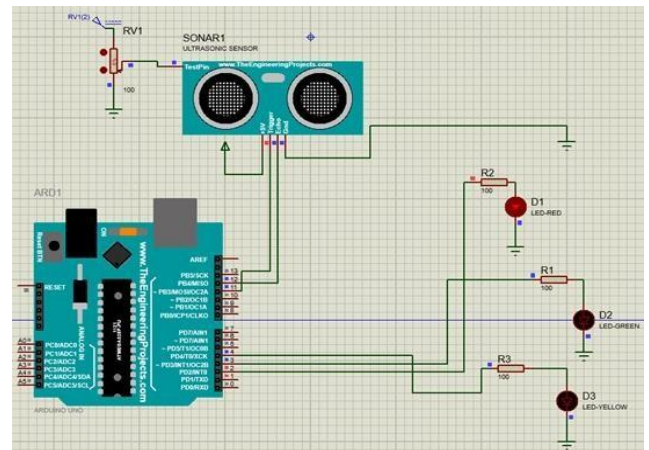


Fig8: One LED ON in Proteus Simulation when the object was far away

Explanation:

In this experiment, two different simulation software were used. Proteus Simulation Software and TinkerCad simulation software both were used to verify the results that were obtained in the experiment. At first, the circuit was made in the Proteus simulation software accordingly. After developing it, the Arduino HEX file was generated from the Arduino IDE and was imported in the proteus simulation software. The results were observed by bringing the cursor close to the sonar. The results were observed by the operation of the LEDs. The same was performed on the TinkerCad simulation software. The circuit was developed on the software. The object was set graphically on the simulation software after importing the Arduino code on the simulation software. The results were observed accordingly as well by observing the LEDs and serial monitor of the Arduino Uno microcontroller that was used on the TinkerCad software.

Discussion

This experiment demonstrated how to build a basic obstacle detection system using an Arduino Uno and an HCSR04 ultrasonic sensor. The sensor accurately measured the distance between itself and nearby objects using ultrasonic waves, and the Arduino processed this data to control LEDs as indicators. Depending on how close the object was, one, two, or all three LEDs turned on, effectively signaling different distance ranges.

The readings were displayed in real-time on the Serial Monitor in both centimeters and inches. This not only confirmed the system's accuracy but also helped verify the correct working of the code and logic implemented. The Arduino's `pulseIn()` function was effective in capturing the echo signal's duration, which was then converted into distance using the speed of sound formula. The threshold-based LED control logic worked reliably in the physical hardware setup.

In addition to the physical setup, simulations were conducted using Proteus and Tinker Cad. Both platforms successfully replicated the system's behavior, providing visual confirmation that the LEDs responded correctly to varying object distances. These simulations validated the design and offered a useful way to test the system before deploying it in real-world applications.

Reference

- [1] "Arduino IDE." [Online]. Available: <https://www.arduino.cc/en/Main/Software>
[Accessed: May 10, 2025].
- [2] "Add and Simulate Ultrasonic Sensor in Proteus - 2018 Edition." Etechnophiles. [Online]. Available: <https://etechnophiles.com/add-simulate-ultrasonic-sensor-proteus-2018-edition/>
[Accessed: May 10, 2025].
- [3] "Ultrasonic Distance Sensor in Arduino With TinkerCad." Instructables. [Online]. Available: <https://www.instructables.com/Ultrasonic-Distance-Sensor-Arduino-Tinkercad/>
[Accessed: May 10, 2025].